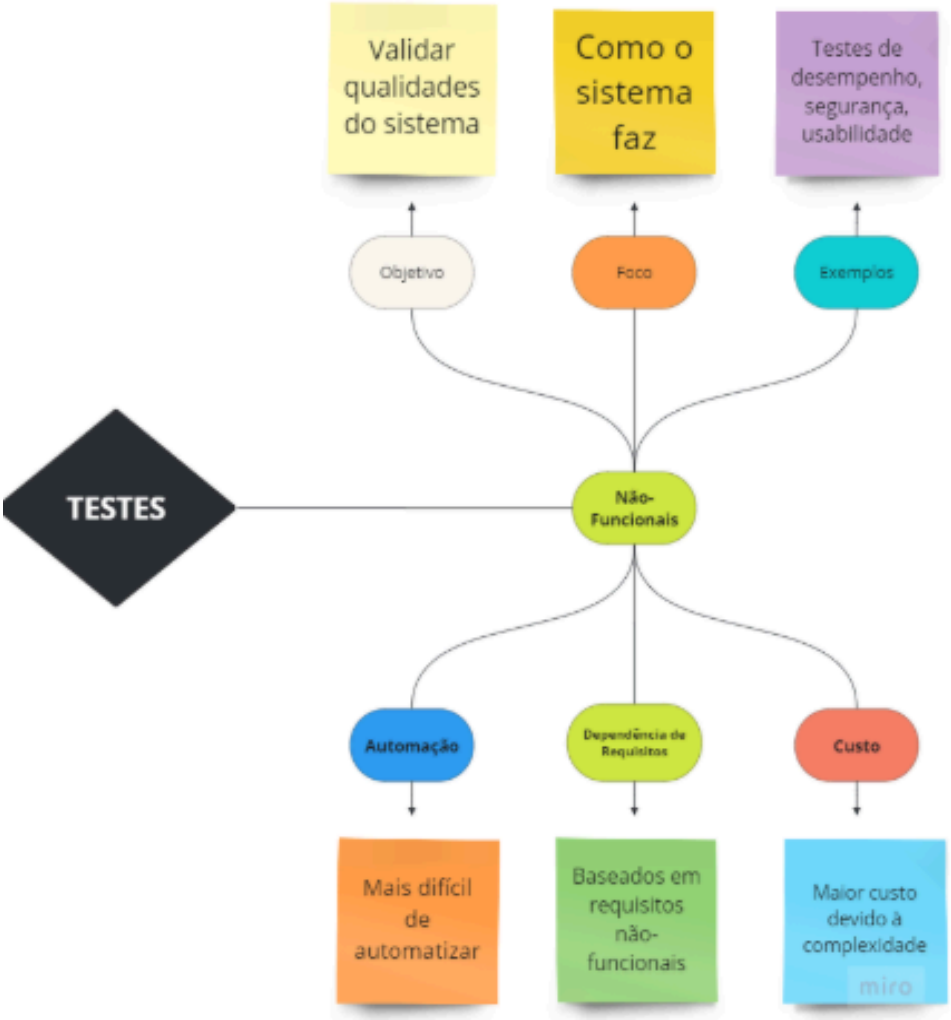
Alunos: Leonardo, Daniel e Geder

Tópico 1 - Testes não Funcionais (Performance, Carga, Estresse)

Os testes não funcionais avaliam características do sistema como desempenho, segurança, usabilidade e confiabilidade, focando em como o sistema se comporta em diferentes condições e não apenas no que ele faz. Eles são importantes porque garantem qualidade geral, boa experiência do usuário e estabilidade da aplicação.

Eles são utilizados para identificar limitações e riscos antes que o sistema seja colocado em produção, sendo realizados por meio de testes específicos como carga, estresse, volume e segurança, simulando cenários reais de uso para analisar o comportamento do sistema.



Perguntas sobre Testes Não Funcionais:

1. Qual a diferença entre testes funcionais e não funcionais, e por que ignorar os testes não funcionais pode comprometer seriamente a qualidade de um sistema?

2. Como você escolheria entre realizar um teste de carga, estresse ou volume em um sistema crítico, e quais critérios utilizaria para essa decisão?

3. De que forma métricas como latência, throughput e tempo de resposta podem indicar gargalos no sistema durante um teste não funcional?

4. Quais são os principais desafios na automação de testes não funcionais e como eles podem ser superados em um ambiente real de desenvolvimento?

5. Como os testes não funcionais contribuem para a experiência do usuário final, mesmo que não estejam diretamente ligados às funcionalidades do sistema?

Exemplos de Testes Não Funcionais:

1. Teste de Desempenho (Carga)
Cenário: Sistema de e-commerce com muitos usuários acessando ao mesmo tempo.
Como Testar: Simular centenas ou milhares de usuários simultâneos realizando ações como login, busca e compra usando ferramentas de teste de carga.
Objetivo: Verificar se o sistema mantém um tempo de resposta aceitável e não apresenta falhas sob alta demanda.

2. Teste de Segurança
Cenário: Aplicação web que armazena dados sensíveis dos usuários.
Como Testar: Realizar tentativas de acesso não autorizado, injeção de SQL e análise de vulnerabilidades com ferramentas específicas.
Objetivo: Garantir que os dados estejam protegidos contra ataques e acessos indevidos.

3. Teste de Usabilidade
Cenário: Usuários utilizando um sistema pela primeira vez.
Como Testar: Observar usuários reais executando tarefas comuns e coletar feedback sobre dificuldade, tempo e erros cometidos.
Objetivo: Avaliar se o sistema é intuitivo, fácil de usar e proporciona uma boa experiência ao usuário.

Desafio Final:

1. Qual é a diferença prática entre avaliar o tempo de resposta em testes de performance, o comportamento no limite esperado em testes de carga e o comportamento até a quebra do sistema em testes de estresse?

Tópico: Usabilidade e Acessibilidade Digital

A. Introdução

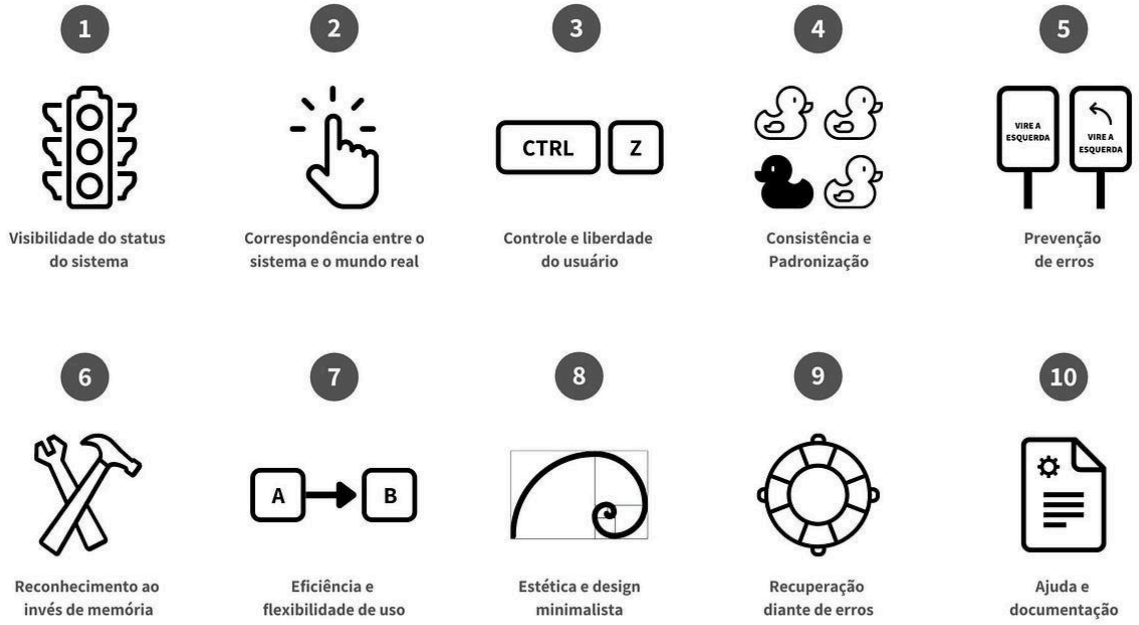
O Porquê: A relevância de um produto digital hoje não reside apenas na sua funcionalidade técnica, mas na experiência que ele proporciona. Em um mercado saturado, a usabilidade é o fator que retém o usuário, reduzindo a frustração e o esforço mental necessário para completar tarefas. Ignorar esses princípios resulta em abandono de plataforma, custos elevados de suporte e uma percepção negativa da marca.

O Como: A aplicação prática ocorre através da implementação das **10 Heurísticas de Nielsen** e das diretrizes **WCAG**. Enquanto a usabilidade foca em tornar a interface intuitiva para o público geral por meio de padrões e feedbacks constantes, a acessibilidade utiliza os princípios **POUR** (Perceptível, Operável, Compreensível e Robusto) para remover barreiras técnicas, garantindo que o software seja utilizável por pessoas com diferentes capacidades motoras, visuais e cognitivas.

B. Visualização Gráfica

Dimensão	Foco Principal	Ferramenta Base	Resultado Esperado
Usabilidade	Eficiência e Satisfação	10 Heurísticas de Nielsen	Fluidez e baixo erro cognitivo.
Acessibilidade	Inclusão e Direito Legal	Princípios WCAG (POUR)	Acesso universal e sem barreiras.

Representação Conceitual: Imagine dois círculos que se cruzam (Diagrama de Venn). No círculo A temos o "Uso Fácil"; no círculo B temos o "Acesso para Todos". Onde eles se cruzam, temos o **Design Inclusivo de Alta Qualidade**.



C. 5 Perguntas Desafiadoras

1. **Paradoxo do Design:** É possível que um sistema tenha uma usabilidade excelente para um usuário padrão, mas seja completamente inacessível para uma pessoa com deficiência visual? Justifique.
2. **Heurística vs. Estética:** De que maneira a Heurística nº 8 (Design Minimalista) pode entrar em conflito com a necessidade de acessibilidade, e como equilibrar ambas?
3. **Carga Cognitiva:** Como a Heurística nº 6 (Reconhecimento em vez de memorização) auxilia especificamente usuários com transtornos de atenção ou deficiências cognitivas leves?
4. **Impacto Legal:** Além da experiência do usuário, quais são as implicações para uma empresa brasileira que ignora a Lei Brasileira de Inclusão (LBI) em seus produtos digitais?
5. **Automação:** Ferramentas automáticas de teste de acessibilidade conseguem garantir 100% de conformidade com a WCAG ou o olhar humano de UX ainda é indispensável?

D. 3 Exemplos de Teste (QA)

Exemplo 1: Teste de Navegação por Teclado

- **Cenário:** Um usuário com deficiência motora tentando preencher um formulário de cadastro.
- **Como Testar:** Tentar navegar por todos os campos e botões utilizando apenas a tecla Tab e submeter o formulário usando Enter.
- **Objetivo:** Validar o princípio **Operável** (WCAG) e garantir que o foco visual esteja sempre nítido.

Exemplo 2: Teste de Feedback de Sistema

- **Cenário:** O usuário clica no botão "Enviar" em um formulário com erro.

- **Como Testar:** Deixar campos obrigatórios vazios e clicar em enviar; observar se a mensagem de erro aparece imediatamente e se indica onde está o erro.
- **Objetivo:** Validar a **Heurística nº 1** (Visibilidade do Status) e a **Heurística nº 9** (Ajuda na recuperação de erros).

Exemplo 3: Teste de Contraste e Legibilidade

- **Cenário:** Um usuário com baixa visão ou daltônico tentando ler as informações de um dashboard.
- **Como Testar:** Utilizar uma ferramenta de inspeção de contraste (como Wave ou Lighthouse) nas cores de fundo e texto.
- **Objetivo:** Validar o princípio **Perceptível** (WCAG), garantindo que a razão de contraste mínima (geralmente 4.5:1) seja atingida.

Tópico 3: Fundamentos de Segurança da Informação nos Testes

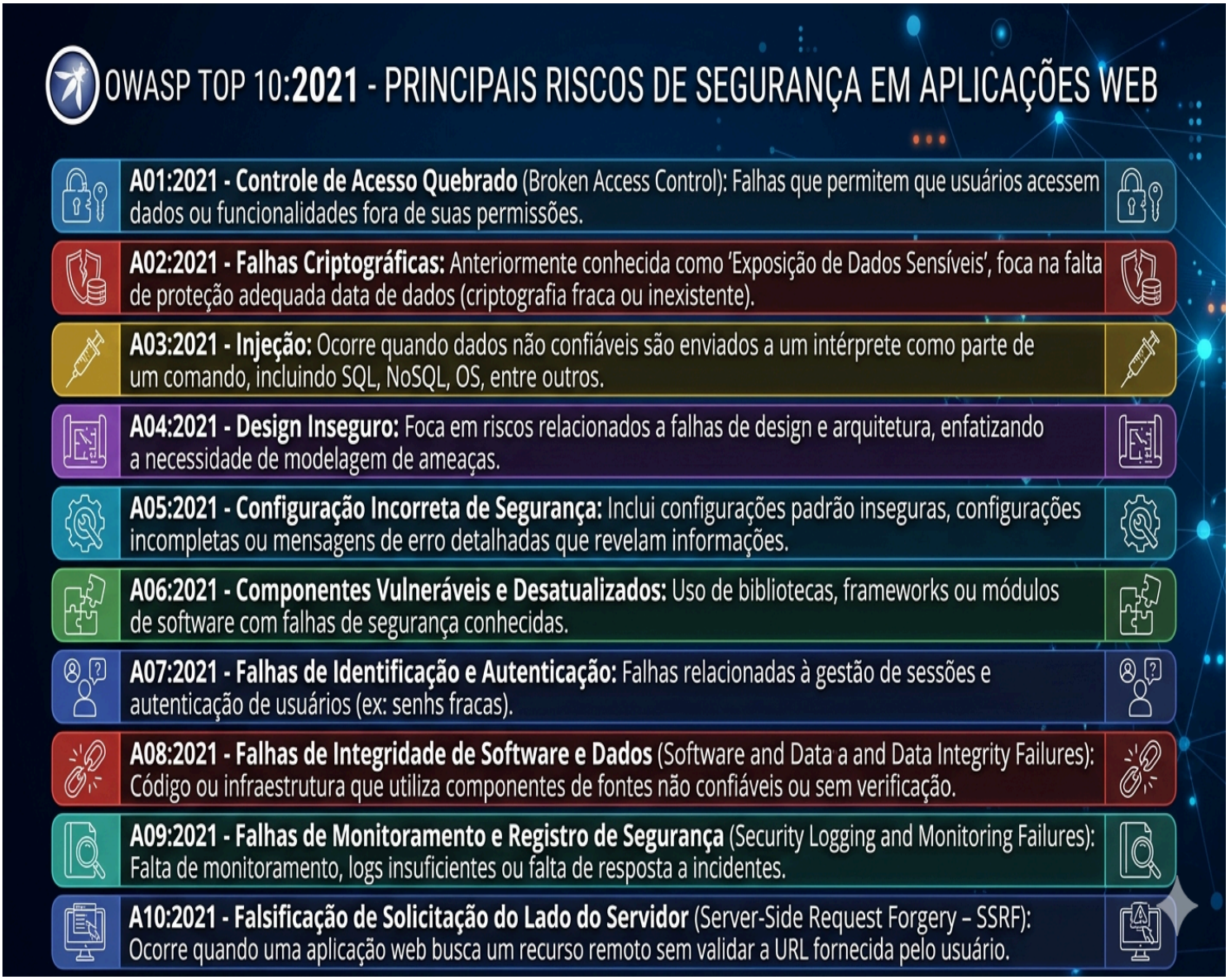
A. Introdução: O "Porquê" e o "Como"

Porquê: O mercado atual exige que um software de excelência faça muito mais do que apenas "funcionar". A garantia da qualidade voltada para segurança (Quality Assurance for Security) existe para garantir que a aplicação proteja os dados sensíveis dos usuários, mantenha autenticações robustas e feche as portas para cibercriminosos. O objetivo não é apenas validar se uma funcionalidade atinge o resultado esperado pelo usuário comum, mas também se ela resiste aos ataques intencionais e maliciosos que acontecem diariamente na web, prevenindo vazamentos de dados que podem destruir a reputação de uma empresa.

Como: Na prática, transformamos a teoria abstrata de segurança em um roteiro testável através do OWASP Top 10, que funciona como um checklist prático dos maiores riscos cibernéticos da atualidade. Testamos a segurança implementando a validação rigorosa de *inputs* (entradas de dados) para evitar injeções de código (como SQL Injection e XSS) e usamos abordagens combinadas: avaliações estáticas do código (SAST), ataques simulados na aplicação em execução (DAST) e o Pentest manual, onde um especialista ativamente procura por falhas lógicas que a automação não consegue detectar.

Resumindo em uma frase, testar a segurança é como conferir se todas as portas, janelas e cofres do nosso sistema estão bem trancados, garantindo que invasores não consigam entrar para roubar ou bagunçar os dados dos nossos usuários.

B. Visualização Gráfica do Tópico



C. 5 Perguntas Desafiadoras sobre o Tópico

Qual é a utilidade do **OWASP Top 10** no dia a dia da equipe de testes?
Qual é a diferença básica entre os testes de segurança **SAST** e **DAST**?
Se já temos ferramentas automáticas que testam o código, por que ainda precisamos de um humano fazendo o **Pentest manual**?
Qual é o perigo de um sistema não fazer a "validação de input" (ou seja, não limpar os dados que o usuário digita nos campos)?
O que um testador está tentando descobrir ao simular um ataque de **SQL Injection** em uma tela de login?

D. 3 Exemplos de Teste

Exemplo 1: Teste contra Injeção de Código (SQL Injection)

Cenário: Formulário de Login da aplicação web.

Como Testar: No campo "Nome de Usuário", digitar o comando admin' OR 1=1 -- e colocar qualquer senha, clicando em Entrar.

Objetivo: Validar se o sistema sanitiza os caracteres especiais do input. O resultado esperado é que a aplicação bloqueia a tentativa, provando que o banco de dados trata o texto apenas como uma String inofensiva e não como uma instrução SQL verdadeira.

Exemplo 2: Teste contra Script Malicioso (XSS - Cross-Site Scripting)

- **Cenário:** Barra de "Pesquisa de Produtos" no topo do site.
- **Como Testar:** Digitar o código `<script>alert('Falha de Segurança')</script>` na barra de pesquisa e apertar Enter.
- **Objetivo:** Garantir que o sistema valida a entrada de dados. O sistema deve mostrar a mensagem "Nenhum produto encontrado", mas **nunca** deve abrir um pop-up na tela (renderizar o código do script). Se o pop-up abrir, significa que o sistema é vulnerável a XSS.

Exemplo 3: Teste de Robustez de Autenticação (Ataque de Força Bruta)

Cenário: Tela de autenticação e recuperação de senha.

Como Testar: Utilizar uma ferramenta de automação para tentar realizar 50 tentativas de login incorretas com senhas aleatórias no período de 10 segundos para a mesma conta.

Objetivo: Validar se o mecanismo de autenticação é robusto. O resultado esperado é que o sistema bloqueia o IP do usuário atacante ou passe a exigir validações adicionais (como um CAPTCHA) após 3 ou 5 tentativas falhas.

Pergunta Desafiadora Escolhida(Leo):

8. O que é o projeto OWASP e por que o seu ranking "OWASP Top 10" é utilizado como a principal referência de mercado para mapear vulnerabilidades críticas em aplicações web?

R:

A OWASP (Open Worldwide Application Security Project) é uma comunidade global focada na segurança de software. O ranking "Top 10" é a principal referência porque ele elenca os riscos cibernéticos mais críticos e reais do mercado atual. Ele funciona para os QAs e desenvolvedores como um "checklist prático", tirando a segurança da teoria abstrata e direcionando o foco exatamente para as portas que os hackers mais costumam usar.